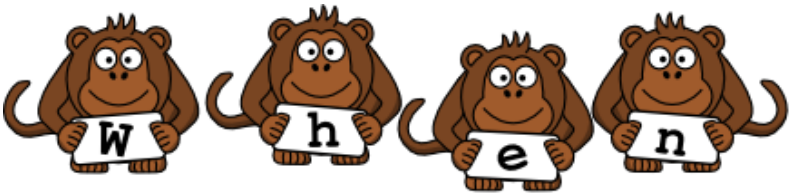
ERHVERVS AKADEMI AARHUS

Shakesperean Monkey Problem.

I denne opgave vil se nærmere på ”*the Shakesperean Monkey Problem*”.



$$\frac{1}{64} \times \frac{1}{64} \times \frac{1}{64} \times \frac{1}{64} = \frac{1}{16{,}777{,}216}$$

Her er aber blevet sat ved nogle skrivemaskiner, og håbet er nu at de kan kæmpe sig frem til at skrive sætningen ”*To be or not to be that is the question*”. Det antages at aberne hver gang de foretager et anslag på tastaturet rammer en tilfældig tast.

Det følger at aberne står foran en stor udfordring, når det her kræves at de skal sammensætte 39 karakterer i den rigtige rækkefølge (for at have løst the” *Shakesperean Monkey Problem*”).

Vi skal nu se på om vi kan hjælpe aberne på vej ved hjælp ved at lave et **Python program**, der på baggrund af en ”genetisk algoritme” og tilfældige strenge (af den korrekte længde, 39, muligvis genereret af aberne, eller af en tilfældigheds generator) er i stand til at generere den korrekte sætning, ”*To be or not to be that is the question*”.

a)

Givet GA terminolog, hvor vi kan kalde en potentiel løsning et ”kromosom”, og hvert element (en karakter) i kromosomet for et ”gen”, hvordan tænker du at en fitness funktion for den genetiske algoritme kunne se ud?

b)

Inden vi går i gang med at lave den genetiske algoritme til at løse problemet, er det godt at have noget at sammenligne med. Så start med at lave en algoritme der laver 10.000, eller flere, tilfældige strenge, og beregner hvilken af dem der har den højeste fitness.

c)

Som bekendt er sådan en metode med tilfældige gæt ikke særlig effektiv, da man ikke drager fordel af gode løsninger man allerede har fundet.

I ”*hill climbing*” starter vi med en tilfældig løsning, og ser på naboer til denne løsning. Vi kan her antage at en nabo til en streng, er en anden streng, der i en eller flere karakterer har en anden værdi (eksperimenter

med forskellige muligheder for begrebet ”nabo”. Kan vi effektivisere søgningen så de ”naboer” vi søger på ikke er dårligere end den løsning vi allerede har fundet osv.).

Udregn fitness for disse naboer, og udvælg den nabo med bedste fitness værdi, og lad denne nabo være udgangspunkt for en fortsat søgning. Eksperimenter med forskellige beskrivelser af begrebet ”nabo”, antal iterationer mm.

d)

Fra opgave b) og c) har vi nu 2 algoritmer at sammenligne vores genetiske algoritme med.

Design nu en genetisk algoritme til at løse problemet, hvor du udvælger passende mutation rates, cross-over operation, størrelse af population, og antal generationer, - der finder, eller kommer tæt på at finde en løsning, givet den tidligere anvendte fitness funktion.

Hvordan klarer den genetiske algoritme sig sammenlignet med random-algoritmen og hill climbing?

